Modelos de embedding e geradores para o RAG offline no Raspberry Pi 4

24 de outubro de 2026 · Isabela

Este documento substitui a lista anterior (SLMs_para_testar_no_Pi4.md). Ele leva em conta o pré-projeto (“Impacto da compressão de modelos de linguagem na qualidade da recuperação da informação em domínio agrícola sob restrições de hardware”), o referencial teórico, a lista `referencias_IC_compressao_LLM.md`, o doc-ranker e os artigos da pasta `Artigos/`. Referências marcadas com 📁 já estão na sua pasta.

Resumo

- **Embedding:** o eixo principal é a família **multilingual-E5** (small, base e large). Você já a cita no referencial (Wang et al., 2022 📁), já a usou no doc-ranker e ela tem três tamanhos com o mesmo treino. Isso permite responder a pergunta central do pré-projeto: vale mais um modelo maior quantizado em INT8/INT4 ou um modelo menor em FP16? Para comparar, eu incluiria o **EmbeddingGemma-300M**, que foi treinado já pensando em quantização, e um modelo nativo em português (**Serafim**).
- **Gerador: Qwen3 1.7B** como principal, **Qwen3 0.6B** como versão barata para os testes de fine-tuning e **Gemma 3 1B** ou **Llama 3.2 1B/3B** como segunda família. O fine-tuning deve ensinar **estilo e fidelidade aos trechos**, não conhecimento agronômico. A literatura mostra que conhecimento novo entra melhor pela recuperação do que pelo fine-tuning.
- **Ordem do pipeline do gerador:** fine-tuning com LoRA/QLoRA no Mac ou no Colab → juntar o adaptador ao modelo → converter para GGUF → quantizar em Q8_0 e Q4_K_M → rodar no Pi. Os níveis de quantização do modelo ajustado viram parte do seu experimento de compressão.

Como os dois modelos se encaixam no pré-projeto

O pré-projeto tem como foco a **qualidade da recuperação** sob compressão (MRR, nDCG@k, Precision@k), e cita três componentes: embedding, re-ranqueador e gerador (seção 2.2). Isso tem duas consequências práticas:

1. **O modelo de embedding é o objeto central do estudo.** Vale escolher modelos que possam ser quantizados de várias formas (FP32 → FP16 → INT8 → INT4) e que existam em vários tamanhos.
2. **O gerador com fine-tuning é uma extensão do escopo.** Ele cabe na seção 2.2 (“o modelo gerador”), mas as métricas de recuperação não avaliam a resposta gerada. Se ele entrar, você vai precisar de métricas de geração (veja a seção “Como avaliar o gerador”) e deve combinar com o orientador se ele é um objetivo específico ou um complemento.

Também vale um ajuste no texto do pré-projeto: ele fala em “consumo de VRAM”, mas o Pi 4 não tem GPU utilizável para isso. No Pi, a medida é a RAM.

Parte 1: Modelo de embedding (recuperação)

O que procurar

Critério	Por que importa no seu caso
Treinado para recuperação (pares consulta–passagem, aprendizado contrastivo)	Modelos treinados só para similaridade de frases (STS/paráfrase) são piores para achar a resposta de uma pergunta. Isso está em Zhao et al. (2024) 📁 e no próprio E5 (Wang et al., 2022) 📁.
Português nos dados de treino e resultado medido em PT	O produtor escreve diferente do livro, e o modelo precisa cobrir esse vocabulário. Benchmarks úteis: mMARCO-PT (que você já usou) e a parte multilíngue do MTEB/MMTEB.
Tamanho e latência por consulta no Pi	No Pi só a consulta é codificada a cada busca (bi-encoder; Xu et al., 2025 📁). Até ~300M parâmetros em INT8 deve responder em tempo aceitável; acima disso, medir.
Dimensão do vetor	Define o tamanho do índice: 20.595 chunks × 384 dimensões × 4 bytes ≈ 32 MB; com 768 dimensões, ≈ 63 MB. Modelos com Matryoshka (MRL) permitem cortar o vetor sem treinar de novo.
Limite de tokens	512 tokens é suficiente para os pares do “500 Perguntas”, com exceção das cerca de 30 respostas longas (já previsto no checklist).
Família com vários tamanhos	Permite separar o efeito da compressão do efeito do tamanho do modelo, que é a hipótese do pré-projeto.
Suporte a ONNX/quantização	Para rodar no Pi sem PyTorch e para gerar as versões INT8/INT4 do seu experimento.
Licença	MIT ou Apache 2.0 facilitam a publicação e a redistribuição do modelo ajustado.

Candidatos

1. multilingual-e5-small / base / large — eixo principal

- **Especificidades:** desenvolvido pela Microsoft Research (Liang Wang, Nan Yang, Furu Wei e outros; publicado no Hugging Face na conta `intfloat`); 118M, 278M e 560M parâmetros; vetores de 384, 768 e 1024 dimensões; base XLM-RoBERTa; exige os prefixos `query:` e `passage:`; licença MIT.
- **Motivação:** é o modelo que você já cita e já testou (no doc-ranker, o base deu MRR@10 de 0,970 no mMARCO-PT com pool pequeno). Os três tamanhos têm o mesmo

treino, o que permite comparar, por exemplo, o *base em INT8* com o *small em FP32*.

- **Prós:** bom em português; muito usado como baseline em artigos; exporta fácil para ONNX; o small cabe com folga no Pi.
- **Contras:** já não é o estado da arte; o large é pesado para o Pi (bom como teto no Mac); esquecer os prefixos derruba a qualidade.
- **Artigos:** Wang et al. (2022), *Text Embeddings by Weakly-Supervised Contrastive Pre-training*, arXiv:2212.03533 📁; Wang et al. (2024), *Multilingual E5 Text Embeddings: A Technical Report*, arXiv:2402.05672.

2. EmbeddingGemma-300M (Google) — o mais interessante para o tema de compressão

- **Especificidades:** ~308M parâmetros; multilíngue (mais de 100 idiomas); vetor de 768 dimensões com Matryoshka (pode ser cortado para 512, 256 ou 128); usa prompts de tarefa no lugar dos prefixos do E5; licença Gemma.
- **Motivação:** foi pensado para rodar em dispositivo e tem versões treinadas com quantização (*quantization-aware training*). Isso permite comparar no seu experimento PTQ e QAT, as duas categorias que o seu referencial descreve (Zhu et al., 2024 📁).
- **Prós:** recente e forte para o tamanho; o Matryoshka adiciona um segundo eixo de compressão (o tamanho do vetor, além dos bits dos pesos).
- **Contras:** maior que o e5-small, então mais lento no Pi; a licença Gemma é menos permissiva que MIT/Apache; o limite de contexto é de 2.048 tokens, o que não é problema aqui.
- **Artigo:** Vera et al. (2025), *EmbeddingGemma: Powerful and Lightweight Text Representations*, arXiv:2509.20354.

3. Serafim PT* (PORTULAN, Universidade de Lisboa) — comparação com um modelo nativo em português

- **Especificidades:** codificadores de frases só para português, em 100M, 335M e 900M parâmetros, com variantes ajustadas para recuperação (`-ir`).
- **Motivação:** mostra se um modelo treinado só em português ganha de um multilíngue no domínio agrícola. É um bom argumento para um trabalho brasileiro.
- **Prós:** vocabulário de português; open source; também tem vários tamanhos.
- **Contras:** o treino puxa mais para o português europeu; tem menos uso e menos ferramentas que o E5; é preciso conferir a licença e o suporte a ONNX de cada variante.
- **Artigo:** Gomes et al. (2024), *Open Sentence Embeddings for Portuguese with the Serafim PT* encoders family*, arXiv:2407.19527.

4. granite-embedding-107m-multilingual (IBM)

- **Especificidades:** 107M parâmetros, 384 dimensões, português entre os idiomas, Apache 2.0.
- **Motivação:** concorrente direto do e5-small no mesmo tamanho e com licença mais livre.
- **Prós:** pequeno e rápido; treinado para recuperação.

- **Contras:** pouco citado em trabalhos acadêmicos; o desempenho em português precisa ser medido por você.
- **Artigo:** Awasthy et al. (2025), *Granite Embedding Models*, arXiv:2502.20204.

5. paraphrase-multilingual-MiniLM-L12-v2 — baseline antigo

- **Especificidades:** 118M parâmetros, 384 dimensões, treinado por destilação para paráfrase.
- **Motivação:** já está no checklist. É útil como "piso", e é um exemplo de destilação (Black-box KD), que aparece no seu referencial.
- **Contras:** não foi treinado para recuperação de pergunta → resposta, então deve perder para o E5.
- **Artigos:** Reimers & Gurevych (2019), *Sentence-BERT*, arXiv:1908.10084; Reimers & Gurevych (2020), *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation*, arXiv:2004.09813.

6. Teto de qualidade (só no Mac, como referência)

- **Qwen3-Embedding-0.6B** (Zhang et al., 2025, arXiv:2506.05176) e **BGE-M3** (Chen et al., 2024, arXiv:2402.03216): estão entre os mais fortes em multilíngue, mas são grandes e lentos para o Pi 4. Servem para mostrar o quanto se perde ao usar um modelo que cabe no Pi.

Re-ranqueador (opcional, como no pré-projeto)

- O mais leve em português é a família de rerankers do **mMARCO** (Unicamp; por exemplo, mMiniLM ajustado no mMARCO). Bonifacio et al. (2021), *mMARCO: A Multilingual Version of the MS MARCO Passage Ranking Dataset*, arXiv:2108.13897. A lista de referências também cita Rosa et al. (2022, arXiv:2212.06121) e Nogueira & Cho (2019).
- No Pi, aplicar só nos top-10 e medir a latência antes de adotar.

Fine-tuning do embedding (se for fazer)

- Os pares pergunta–resposta do "500 Perguntas" são dados prontos para treino contrastivo.
- **Cuidado com vazamento:** o golden set sai dos mesmos livros. Separe treino e teste **por livro ou cultura** (por exemplo, treinar com 30 livros e testar com os outros), e não por pergunta.
- Para gerar consultas no estilo do produtor sem anotar à mão, há duas técnicas de autores brasileiros: InPars (Bonifacio et al., 2022, arXiv:2202.05144) e GPL (Wang et al., 2022, arXiv:2112.07577).

Parte 2: Modelo gerador (com fine-tuning)

O que procurar

Critério	Por que importa no seu caso
Cabe no Pi depois de quantizado	Com 4 GB de RAM, o limite prático é ~3–4B em Q4. Abaixo de 2B sobra memória para o embedding e o índice.
Português e tokenizador eficiente em PT	Menos tokens por frase deixa a leitura do prompt mais rápida, e esse é o gargalo no Pi.
Segue instruções e se apoia nos trechos	Em RAG, importa mais usar o contexto e dizer "não sei" do que "saber" agronomia.
Facilidade de fine-tuning	Suporte em PEFT/QLoRA, Unsloth ou MLX-LM (no Mac), e uma versão *instruct* com template de chat estável.
Conversão para GGUF no llama.cpp	Arquitetura suportada; caso contrário, o modelo ajustado não roda no Pi.
Licença	Precisa permitir fine-tuning e publicação do modelo derivado (Apache 2.0 é o mais simples).
Sensibilidade à quantização	Modelos pequenos perdem proporcionalmente mais em 4 bits. Husom et al. (2025) 📁 e *Quantization Hurts Reasoning* 📁 dão apoio para medir isso, em vez de supor.

Candidatos

1. Qwen3 1.7B — principal

- **Especificidades:** ~1,7B parâmetros (Q4_K_M ≈ 1,1 GB); multilíngue com português; Apache 2.0; modo de raciocínio que pode ser desligado.
- **Motivação:** melhor equilíbrio entre tamanho e qualidade nessa faixa, ferramentas maduras para fine-tuning (Unsloth, MLX-LM, PEFT) e suporte direto no llama.cpp.
- **Prós:** tem a versão 0.6B, da mesma família, para testes rápidos e para comparar o efeito do tamanho; a licença permite publicar o modelo ajustado.
- **Contras:** no fine-tuning é preciso usar o template sem raciocínio (`enable_thinking=False`), senão o modelo aprende a gerar blocos `<think>` que deixam tudo lento no Pi; às vezes mistura termos em inglês.
- **Artigo:** Qwen Team (2025), *Qwen3 Technical Report*, arXiv:2505.09388.

2. Qwen3 0.6B — laboratório de fine-tuning

- **Motivação:** o fine-tuning roda em minutos, então dá para testar formatos de dados e hiperparâmetros antes de passar para o 1.7B. Também mostra até onde um modelo muito pequeno chega depois do ajuste.
- **Contras:** sem ajuste, tende a copiar ou embaralhar o trecho; mesmo depois do ajuste, a qualidade é limitada.

3. Gemma 3 1B (Google) — segunda família

- **Especificidades:** ~1B (Q4 ≈ 0,8 GB); tokenizador grande e multilíngue (bom para português); licença Gemma.
- **Motivação:** comparar uma segunda família evita que a conclusão dependa só do Qwen.
- **Prós:** muito leve e rápido no Pi; boas ferramentas de fine-tuning.
- **Contras:** o suporte multilíngue amplo é do 4B para cima, e o 1B foca mais em inglês; a licença Gemma tem regras de uso próprias.
- **Artigo:** Gemma Team (2025), *Gemma 3 Technical Report*, arXiv:2503.19786.

4. Llama 3.2 1B / 3B Instruct (Meta)

- **Motivação:** é o modelo mais usado em estudos com Raspberry Pi e edge, o que facilita comparar seus números com a literatura. O português é um dos idiomas oficialmente suportados.
- **Prós:** muita documentação de fine-tuning; o 3B é o teto que ainda roda no Pi.
- **Contras:** licença Llama Community (exige atribuição e tem restrições); o 3B leva de 1 a 3 minutos para ler um prompt de RAG no Pi 4 (estimativa, precisa ser medida).
- **Artigo:** Grattafiori et al. (2024), *The Llama 3 Herd of Models*, arXiv:2407.21783.

5. Tucano (1.1B / 2.4B) — opcional, nativo em português

- **Motivação:** pré-treinado do zero em português brasileiro. Testar se um modelo nativo em PT, depois de ajustado, alcança um multilíngue é uma pergunta de pesquisa interessante.
- **Contras:** treinado com bem menos dados; segue instruções de forma mais fraca; pode exigir conversão manual para GGUF.
- **Artigo:** Corrêa et al. (2024), *Tucano: Advancing Neural Text Generation for Portuguese*, arXiv:2411.07854.

Não recomendo para o Pi 4

- **Modelos de 7–8B:** em Q4 passam de 4 GB e não cabem sem swap no microSD.
- **Phi-4-mini (3,8B):** o português é mais fraco e ele fica no limite da RAM.

Como fazer o fine-tuning “no estilo Embrapa”

O que ensinar. O estilo do “500 Perguntas” é resposta curta, direta e técnica, para o produtor. Ovadia et al. (2023, arXiv:2312.05934) mostram que, para colocar conhecimento novo no modelo, RAG funciona melhor que fine-tuning. Gekhman et al. (2024, arXiv:2405.05904) mostram que ajustar com conhecimento que o modelo não tinha aumenta a alucinação. Por isso o ajuste deve ensinar **formato, tom e fidelidade aos trechos**, e o conhecimento deve continuar vindo da recuperação.

Formato dos dados (inspirado no RAFT). Zhang et al. (2024), *RAFT: Adapting Language Model to Domain Specific RAG*, arXiv:2403.10131, sugerem treinar com o mesmo formato do uso real: - entrada: instrução + pergunta (reescrita no jeito do produtor) + 3 trechos recuperados, sendo um o correto e os outros distratores; - saída: a resposta do livro, citando o número da pergunta; - em parte dos exemplos, o trecho correto fica de fora e a saída ensinada é “não encontrei isso nos documentos”. Isso treina o “não sei”.

Evidência no domínio agrícola. Balaguer et al. (2024), *RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture*, arXiv:2401.08406, comparam as duas abordagens em agricultura e relatam ganhos somados ao combiná-las. É a referência mais próxima do seu caso.

Técnica. LoRA (Hu et al., 2021, arXiv:2106.09685) ou QLoRA (Dettmers et al., 2023, arXiv:2305.14314 📁). No Mac, com MLX-LM; no Colab, com Unsloth/PEFT. Modelos de até 2B treinam com folga num Mac com 16 GB.

Separação dos dados. Use a mesma regra do embedding: treino e teste separados **por livro ou cultura**. Se o modelo ajustado vir no treino as perguntas do teste, a avaliação mede memorização.

Gerar perguntas reescritas. Usar um modelo maior no Mac (ou via API) para reescrever as perguntas é uma forma de destilação Black-box KD, que já está no seu referencial (Zhu et al., 2024 📁; Xu et al., 2024, *Survey on KD of LLMs* 📁).

Quantização depois do ajuste. Juntar o adaptador LoRA ao modelo → converter para GGUF → quantizar em Q8_0, Q5_K_M e Q4_K_M → comparar. Isso liga o fine-tuning ao seu tema de compressão: o ajuste sobrevive à quantização em 4 bits? Os artigos de PTQ da sua pasta (GPTQ 📁, AWQ 📁, SmoothQuant 📁) dão o fundamento. No Pi, o formato mais prático é o K-quant do llama.cpp. Como alternativa de QAT, há o QA-LoRA (já na sua lista de referências).

Como avaliar o gerador

- **Fidelidade e relevância:** RAGAS (Es et al., 2023, arXiv:2309.15217) mede se a resposta está apoiada nos trechos e se responde à pergunta.
- **Juiz automático:** um modelo maior no Mac julga correção e estilo (Zheng et al., 2023, *Judging LLM-as-a-Judge*, arXiv:2306.05685). Valide o juiz com uma amostra anotada por você, seguindo a escala de três níveis do pré-projeto.
- **Pi:** tempo até o primeiro token, tokens/s e RAM de pico, para cada nível de quantização.
- **Posição dos trechos:** Liu et al. (2023), *Lost in the Middle*, arXiv:2307.03172, mostram que o modelo usa pior o que fica no meio do contexto. Mais um motivo para testar top-3 e top-5.
- **Tamanho dos chunks:** os dois artigos de chunking da sua pasta 📁 ajudam a justificar o chunk = par pergunta–resposta.

Matriz de experimentos sugerida

Componente	Modelos	Compressão	Métricas
Embedding	e5-small, e5-base, (e5-large no Mac), EmbeddingGemma-300M, Serafim-IR, MiniLM (piso)	FP32, FP16, INT8 (ONNX), INT4; MRL 768→256 no EmbeddingGemma; vetores em float32, int8 e binário	Recall@k, MRR@10, nDCG@10, Precision@k; latência e RAM no Pi; tamanho do índice

Componente	Modelos	Compressão	Métricas
Gerador	Qwen3 0.6B e 1.7B (base e ajustado), Gemma 3 1B ou Llama 3.2 1B	Q8_0, Q5_K_M, Q4_K_M	RAGAS, juiz validado, “não sei” nas perguntas fora do escopo; tempo até o primeiro token, tokens/s, RAM

Observação sobre “vetores em int8 e binário”: além de quantizar os pesos do modelo, dá para quantizar os próprios vetores do índice. É outro eixo de compressão, barato de testar e diretamente ligado à sua hipótese de que a qualidade do embedding é sensível à compressão.

Nota sobre licenças

- **MIT/Apache 2.0** (E5, Granite, Qwen3, SmolLM3): é preciso só manter o aviso de copyright e o texto da licença, sem restrições de uso. O Apache 2.0 ainda inclui uma licença de patentes.
- **Gemma** (EmbeddingGemma, Gemma 3): permite pesquisa, uso comercial, fine-tuning e publicação do modelo ajustado, mas:
  - há uma Política de Uso Proibido, que vale para você e para quem usar o modelo;
  - ao publicar um derivado, é preciso incluir os termos e avisar que ele vem do Gemma; o derivado herda as mesmas regras e não pode ser relicenciado, por exemplo, como MIT;
  - pelos termos, até um modelo treinado com as saídas do Gemma (destilação) conta como derivado;
  - o Google pode atualizar os termos e restringir usos que considere violação;
  - a OSI não reconhece a licença como open source; no artigo, o termo correto é “pesos abertos”.
- **Llama Community** (Llama 3.2): também exige atribuição e tem restrições de uso.
- Para a IC, nenhuma dessas licenças impede o trabalho. Antes de publicar, confira o texto atual (para o Gemma, em ai.google.dev/gemma/terms). Esta nota não é parecer jurídico.

Referências que você ainda não tem

Wang et al. 2024 (mE5, 2402.05672) · Vera et al. 2025 (EmbeddingGemma, 2509.20354) · Gomes et al. 2024 (Serafim, 2407.19527) · Awasthy et al. 2025 (Granite Embedding, 2502.20204) · Reimers & Gurevych 2019/2020 (1908.10084, 2004.09813) · Zhang et al. 2025 (Qwen3-Embedding, 2506.05176) · Chen et al. 2024 (BGE-M3, 2402.03216) · Bonifacio et al. 2021 (mMARCO, 2108.13897) · Bonifacio et al. 2022 (InPars, 2202.05144) · Wang et al. 2022 (GPL, 2112.07577) · Qwen Team 2025 (Qwen3, 2505.09388) · Gemma Team 2025 (Gemma 3, 2503.19786) · Grattafiori et al. 2024 (Llama 3, 2407.21783) · Corrêa et al. 2024 (Tucano, 2411.07854) · Hu et al. 2021 (LoRA, 2106.09685) · Zhang et al. 2024 (RAFT, 2403.10131) · Ovadia et al. 2023 (2312.05934) · Gekhman et al. 2024 (2405.05904) · Balaguer et al. 2024 (2401.08406) · Es et al. 2023 (RAGAS, 2309.15217) · Zheng et al. 2023 (2306.05685) · Liu et al. 2023 (2307.03172).

Conferi na web os números arXiv de EmbeddingGemma, Granite Embedding e Serafim. Os demais são de memória: confira os números e os primeiros autores antes de citar. Tamanhos em GB e tempos no Pi são estimativas e precisam ser medidos.